

串或字符串 (**string**) 属于线性结构, 自然地可直接利用向量或列表等序列结构加以实现。但字符串作为数据结构, 特点也极其鲜明, 这可归纳为: 结构简单, 规模庞大, 元素重复率高。

所谓结构简单, 是指字符表本身的规模不大, 甚至可能极小。以生物信息序列为例, 参与蛋白质 (文本) 合成的常见氨基酸 (字符) 只有**20**种, 而构成DNA序列 (文本) 的碱基 (字符) 仅有**4**种。尽管就规模而言, 地球系统模式的单个输出文件长达**1~100GB**, 微软Windows系统逾**4000**万行的源代码长度累计达到**40GB**, 但它们都只不过是**ASCII**字符, 甚至是可打印字符组成的。因此, 以字符串形式表示的海量文本数据的高效处理技术, 一直都是相关领域的研究重点。

鉴于字符串结构的上述特点, 本章将直接利用C++本身所提供的字符数组, 并转而将讲述的重点, 集中于各种串匹配算法**indexOf()**的基本原理与高效实现。

§ 11.1 串及串匹配

11.1.1 串

■ 字符串

一般地, 由**n**个字符构成的串记作:

$$S = "a_0 a_1 \dots a_{n-1}", \quad \text{其中, } a_i \in \Sigma, \quad 0 \leq i < n$$

这里的 **Σ** 是所有可用字符的集合, 称作字符表 (**alphabet**), 例如二进制比特集 $\Sigma = \{0, 1\}$ 、ASCII字符集、Unicode字符集、构成DNA序列的所有碱基、组成蛋白质的所有氨基酸, 等等。

字符串**S**所含字符的总数**n**, 称作**S**的长度, 记作 $|S| = n$ 。这里只考虑长度有限的串, $n < \infty$ 。特别地, 长度为零的串称作空串 (**null string**)。请注意, 空串并非由空格字符'□'组成的串, 二者完全不同。

■ 子串

字符串中任一连续的片段, 称作其子串 (**substring**)。具体地, 对于任意的 $0 \leq i \leq i + k < n$, 由字符串**S**中起始于位置**i**的连续**k**个字符组成的子串记作:

$$S.\text{substr}(i, k) = "a_i a_{i+1} \dots a_{i+k-1}" = S[i, i + k)$$

有两种特殊子串: 起始于位置**0**、长度为**k**的子串称为前缀 (**prefix**), 而终止于位置**n - 1**、长度为**k**的子串称为后缀 (**suffix**), 分别记作:

$$\text{prefix}(S, k) = S.\text{substr}(0, k) = S[0, k)$$

$$\text{suffix}(S, k) = S.\text{substr}(n - k, k) = S[n - k, n)$$

由上述定义可直接导出以下结论: 空串是任何字符串的子串, 也是任何字符串的前缀和后缀; 任何字符串都是自己的子串, 也是自己的前缀和后缀。此类子串、前缀和后缀分别称作平凡子串 (**trivial substring**)、平凡前缀 (**trivial prefix**) 和平凡后缀 (**trivial suffix**)。反之, 字符串本身之外的所有非空子串、前缀和后缀, 分别称作真子串 (**proper substring**)、真前缀 (**proper prefix**) 和真后缀 (**proper suffix**)。

■ 判等

最后，字符串 $S[0, n)$ 和 $T[0, m)$ 称作相等，当且仅当二者长度相等（ $n = m$ ），且对应的字符分别相同（对任何 $0 \leq i < n$ 都有 $S[i] = T[i]$ ）。

■ ADT

串结构主要的操作接口可归纳为表11.1。

表11.1 串ADT支持的操作看接口

操 作 接 口	功 能
length()	查询串的长度
charAt(i)	返回第i个字符
substr(i, k)	返回从第i个字符起、长度为k的子串
prefix(k)	返回长度为k的前缀
suffix(k)	返回长度为k的后缀
equal(T)	判断T是否与当前字符串相等
concat(T)	将T串接在当前字符串之后
indexOf(P)	若P是当前字符串的一个子串，则返回该子串的起始位置；否则返回-1

比如，依次对串 $S = \text{"data structures"}$ 执行如下操作，结果依次如表11.2所示。

表11.2 串操作实例

操 作	输 出	字 符 串 s
length()	15	"data structures"
charAt(5)	's'	"data structures"
prefix(4)	"data"	"data structures"
suffix(10)	"structures"	"data structures"
concat("and algorithms")	"data structures and algorithms"	
equal("data structures")	false	"data structures and algorithms"
equal("data structures and algorithms")	true	"data structures and algorithms"
indexOf("string")	-1	"data structures and algorithms"
indexOf("algorithm")	20	"data structures and algorithms"

11.1.2 串匹配

■ 应用与问题

在涉及字符串的众多实际应用中，模式匹配是最常使用的一项基本操作。比如UNIX Shell的grep工具（General Regular Expression Parser）和DOS的find命令，基本功能都是在指定的字符串中查找^①特定模式的字符串。又如生物信息处理领域，也经常需要在蛋白质序列中

^① 这两个命令都是以文件形式来指定待查找的主串，具体格式分别是：

```
% grep <pattern> <file>
c:\> find "pattern" <file>
```

寻找特定的氨基酸模式，或在DNA序列中寻找特定的碱基模式。再如，邮件过滤器也需根据事先定义的特征串，通过扫描电子邮件的地址、标题及正文来识别垃圾邮件。还有，反病毒系统也会扫描刚下载的或将要执行的程序，并与事先提取的特征串相比对，以判定其中是否含有病毒。

上述所有应用问题，本质上都可转化和描述为如下形式：

如何在字符串数据中，检测和提取以字符串形式给出的某一局部特征

这类操作都属于串模式匹配（string pattern matching）范畴，简称串匹配。一般地，即：

对基于同一字符表的任何主串 T （ $|T| = n$ ）和模式串 P （ $|P| = m$ ）：

判定 T 中是否存在某一子串与 P 相同

若存在（匹配），则报告该子串在 T 中的起始位置

串的长度 n 和 m 本身通常都很大，但相对而言 n 更大，即满足 $2 \ll m \ll n$ 。比如，若：

$T = \text{"Now is the time for all good people to come"}$

$P = \text{"people"}$

则匹配的位置应该是 $T.\text{indexOf}(P) = 29$ 。

■ 问题分类

根据具体应用的要求不同，串匹配问题可以多种形式呈现。

有些场合属于模式检测（pattern detection）问题：我们只关心是否存在匹配而不关心具体的匹配位置，比如垃圾邮件的检测。有些场合属于模式定位（pattern location）问题：若经判断的确存在匹配，则还需确定具体的匹配位置，比如带毒程序的鉴别与修复。有些场合属于模式计数（pattern counting）问题：若有多处匹配，则统计出匹配子串的总数，比如网络热门词汇排行榜的更新。有些场合则属于模式枚举（pattern enumeration）问题：在有多处匹配时，报告出所有匹配的具体位置，比如网络搜索引擎。

11.1.3 测评标准与策略

串模式匹配是一个经典的问题，有名字的算法已不下三十种。鉴于串结构自身的特点，在设计和分析串模式匹配算法时也必须做特殊的考虑。其中首先需要回答的一个问题就是，如何对任一串匹配算法的性能作出客观的测量和评估。

多数读者首先会想到采用评估算法性能的常规口径和策略：以时间复杂度为例，假设主串 T 和模式串 P 都是随机生成的，然后综合其各种组合从数学或统计等角度得出结论。很遗憾，此类构思并不适用于这一问题。

以基于字符表 $\Sigma = \{0, 1\}$ 的二进制串为例。任给长度为 n 的主串，其中长度为 m 的子串不过 $n - m + 1$ 个（ $m \ll n$ 时接近于 n 个）。另一方面，长度为 m 的随机模式串多达 2^m 个，故匹配成功的概率为 $n / 2^m$ 。以 $n = 100,000$ 、 $m = 100$ 为例，这一概率仅有

$$100,000 / 2^{100} < 10^{-25}$$

对于更长的模式串、更大的字符表，这一概率还将更低。因此，这一策略并不能有效地覆盖成功匹配的情况，所得评测结论也无法准确地反映算法的总体性能。

实际上，有效涵盖成功匹配情况的一种简便策略是，随机选取主串 T ，并从 T 中随机取出长度为 m 的子串作为模式串 P 。这也是本章将采用的评价标准。

§ 11.2 蛮力算法

11.2.1 算法描述

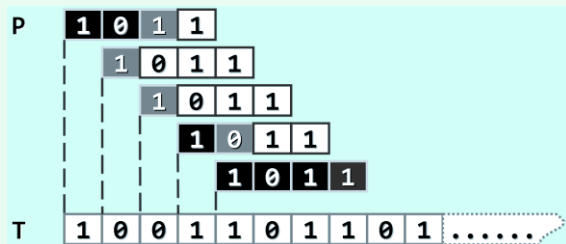


图11.1 串模式匹配的蛮力算法

蛮力串匹配是最直接最直觉的方法。如图11.1所示，可假想地将主串和模式串分别写在两条印有等间距方格的纸带上，主串对应的纸带固定，模式串纸带的首字符与主串纸带的首字符对齐，二者都沿水平方向放置。于是，只需将P与T中长度为m的 $n - m + 1$ 个子串逐一比对，即可确定可能的匹配位置。

不妨按自左向右的次序考查各子串。在初始状态下，T的前m个字符将与P的m个字符两两对齐。接下来，自左向右检查相互对齐的这m对字符：若当前字符对相互匹配，则转向下一对字符；反之一旦失配，则说明在此位置主串与模式串不可能完全匹配，于是可将P对应的纸带右移一个字符，然后从其首字符开始与T中对应的新子串重新对比。图中，模式串P的每一黑色方格对应于字符对的一次匹配，每一灰色方格对应于一次失配，白色方格则对应于未进行的一次比对。若经过检查，当前的m对字符均匹配，则意味着整体匹配成功，从而返回匹配子串的位置。

蛮力算法的正确性显而易见：既然只有在某一轮的m次比对全部成功之后才成功返回，故不致于误报；反过来，所有对齐位置都会逐一尝试，故亦不致漏报。

11.2.2 算法实现

以下给出蛮力算法的两个实现版本。二者原理相同、过程相仿，但分别便于引入后续的不同改进算法，故在此先做一比较。

```

1  /*****
2  * Text      :  0  1  2  .  .  .  i-j  .  .  .  i  .  .  n-1
3  *
4  * Pattern   :
5  *
6  *****/
7  int match(char* P, char* T) { //串匹配算法 (Brute-force-1)
8      size_t n = strlen(T), i = 0; //主串长度、当前接受比对字符的位置
9      size_t m = strlen(P), j = 0; //模式串长度、当前接受比对字符的位置
10     while (j < m && i < n) //自左向右逐个比对字符
11         if (T[i] == P[j]) //若匹配
12             { i++; j++; } //则转到下一对字符
13         else //否则
14             { i -= j - 1; j = 0; } //主串回退、模式串复位
15     return i - j; //如何通过返回值，判断匹配结果？
16 }
```

代码11.1 蛮力串匹配算法（版本一）